Exporting & Importing of Schema / Tables

on AWS RDS using DBMS_DATAPUMP & S3

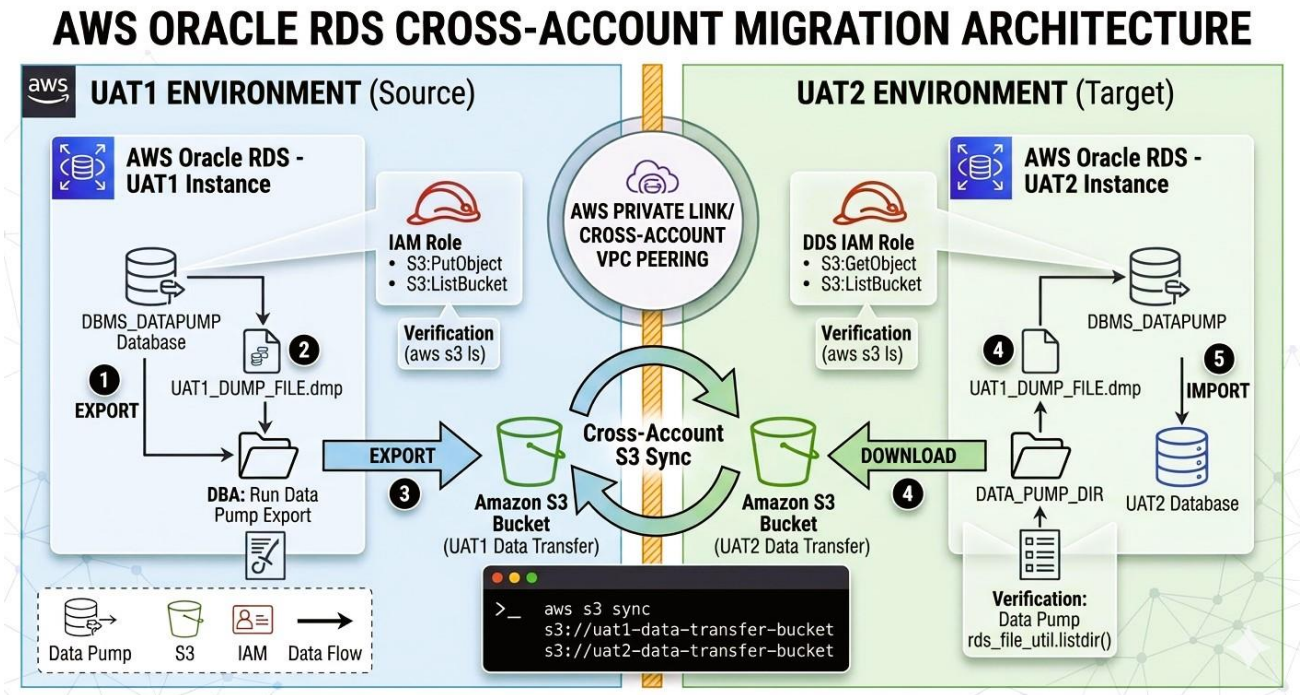
A Complete Step-by-Step Guide with Common Errors & Fixes

Working with Oracle on AWS RDS brings a unique constraint: you do not have OS-level access. This means you cannot run the classic expdp / impdp commands from a terminal. Instead, AWS provides the DBMS_DATAPUMP PL/SQL package and the rdsadmin.rdsadmin_s3_tasks utility to handle data movement entirely from within SQL sessions.

This guide walks you through the complete workflow of exporting a single table, uploading it to S3, downloading it on a target RDS instance, and importing it — along with every common error you may encounter and its fix.

Complete AWS Architecture Flow

The diagram below shows the full end-to-end journey of your data — from the source RDS instance through S3 to the target RDS instance, and the IAM role that authorises each step.



1. Prerequisites & Setup

1.1 Verify DATA_PUMP_DIR

AWS RDS automatically provides a pre-configured Oracle Directory object called DATA_PUMP_DIR. You do not need to create it. Verify it exists:

```
SELECT directory_name,
directory_path
FROM   all_directories
WHERE  directory_name = 'DATA_PUMP_DIR';
```

1.2 List Files in DATA_PUMP_DIR

```
SELECT *
FROM   TABLE(rdsadmin.rds_file_util.listdir('DATA_PUMP_DIR'))
ORDER  BY filename;
```

1.3 Verify the IAM Role Attached to Your RDS Instance

```
# Replace <YOUR_RDS_INSTANCE> and <YOUR_AWS_REGION>
aws rds describe-db-instances \
  --db-instance-identifier <YOUR_RDS_INSTANCE> \
  --region <YOUR_AWS_REGION> \
  --query 'DBInstances[0].AssociatedRoles' \
  --output table
```

The role must appear with FeatureName = S3_INTEGRATION and Status = ACTIVE.

1.4 Confirm the IAM Role ARN

```
# Get the exact ARN — copy this for upload_to_s3 calls
aws iam get-role \
  --role-name <YOUR_IAM_ROLE_NAME> \
  --query 'Role.Arn' \
  --output text

# Expected output format:
# arn:aws:iam::<YOUR_AWS_ACCOUNT_ID>:role/<YOUR_IAM_ROLE_NAME>
```

1.5 Required IAM Policy

Attach this policy to the IAM role. Replace the bucket name placeholder:

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Action": [
      "s3:PutObject",
      "s3:GetObject",
      "s3:ListBucket",
      "s3:DeleteObject"
    ],
    "Resource": [
      "arn:aws:s3:::<YOUR_S3_BUCKET>",
      "arn:aws:s3:::<YOUR_S3_BUCKET>/*"
    ]
  }]
}
```

The trust policy must allow rds.amazonaws.com to assume the role:

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": { "Service": "rds.amazonaws.com" },
    "Action": "sts:AssumeRole"
  }]
}
```

2. Exporting a Single Table or Schema

2.1 Key Difference from Schema Export

Setting	Schema Export	Single Table Export
job_mode	'SCHEMA'	'TABLE'
METADATA_FILTER	SCHEMA_EXPR only	SCHEMA_EXPR + NAME_EXPR
Scope	Entire schema	Named table(s) only
Typical use	Full migration / backup	Selective data transfer

```
DECLARE
  v_hdnl NUMBER;
BEGIN
  v_hdnl := DBMS_DATAPUMP.OPEN(
    operation => 'EXPORT',
    job_mode  => 'TABLE',    -- KEY: TABLE or SCHEMA based on your requirement.
    job_name  => null
  );

  -- Dump file
  DBMS_DATAPUMP.ADD_FILE(
    handle    => v_hdnl,
    filename  => '<YOUR_DUMP_FILENAME>',
    directory => 'DATA_PUMP_DIR',
    filetype  => dbms_datapump.ku$_file_type_dump_file
  );

  -- Log file
  DBMS_DATAPUMP.ADD_FILE(
    handle    => v_hdnl,
    filename  => '<YOUR_LOG_FILENAME>',
    directory => 'DATA_PUMP_DIR',
    filetype  => dbms_datapump.ku$_file_type_log_file
  );

  -- Filter to your schema
  DBMS_DATAPUMP.METADATA_FILTER(
    handle => v_hdnl,
    name   => 'SCHEMA_EXPR',
    value  => 'IN (''<SOURCE_SCHEMA>'')'
  );

  -- Filter to your specific table
  DBMS_DATAPUMP.METADATA_FILTER(
    handle => v_hdnl,
    name   => 'NAME_EXPR',
    value  => 'IN (''<YOUR_TABLE_NAME>'')'
  );

  DBMS_DATAPUMP.START_JOB(v_hdnl);
END;
/
```

2.3 Export Multiple Tables in One Shot

```
-- Extend the IN list to include more tables
DBMS_DATAPUMP.METADATA_FILTER(
```

```
  handle => v_hdnl,
  name   => 'NAME_EXPR',
  value  => 'IN (''<TABLE_1>'', ''<TABLE_2>'', ''<TABLE_3>'')'
);
```

2.4 Pro Tip: Dynamic Timestamp in Filename

Prevents overwriting a previous dump when you re-run the same export.

```
DECLARE
  v_hdnl    NUMBER;
  v_dmpfile VARCHAR2(100);
BEGIN
  -- e.g. produces: MYTABLE_20260318_1045.dmp
  v_dmpfile := '<YOUR_TABLE_NAME>_'
               || TO_CHAR(SYSDATE, 'YYYYMMDD_HH24MI') || '.dmp';

  v_hdnl := DBMS_DATAPUMP.OPEN(
    operation => 'EXPORT',
    job_mode  => 'TABLE',
    job_name  => null
  );
  DBMS_DATAPUMP.ADD_FILE(
    handle    => v_hdnl,
    filename  => v_dmpfile,
    directory => 'DATA_PUMP_DIR',
    filetype  => dbms_datapump.ku$_file_type_dump_file
  );
  -- ... add log, filters, then START_JOB
END;
/
```

2.5 Monitor the Export Job

START_JOB is asynchronous. Always verify it finished before uploading to S3:

```
-- Check if job is still running
SELECT job_name, operation, job_mode, state, degree
FROM   dba_datapump_jobs
WHERE  state = 'EXECUTING';

-- Read the log file to confirm success
SELECT text
FROM   TABLE(
  rdsadmin.rds_file_util.read_text_file(
```

```
    p_directory => 'DATA_PUMP_DIR',
    p_filename  => '<YOUR_LOG_FILENAME>'
  )
);
```

3. Uploading the Dump File to S3

3.1 How p_prefix Matching Works

p_prefix Value	Files Uploaded	Safe?
'<YOUR_DUMP_FILENAME>'	Only the exact .dmp file	YES — safest
'<YOUR_TABLE_NAME>_'	All files starting with that prefix	YES — .dmp + .log
'' (empty string)	ALL files in DATA_PUMP_DIR	DANGEROUS

Pro tip: Upload to S3 only what is required.

3.2 Upload Block

```
BEGIN
  rdsadmin.rdsadmin_s3_tasks.upload_to_s3(
    p_bucket_name    => '<YOUR_S3_BUCKET>',
    p_directory_name => 'DATA_PUMP_DIR',
    p_s3_prefix      => '<YOUR_S3_PREFIX>',   -- e.g. 'exports/'
    p_prefix         => '<YOUR_DUMP_FILENAME>',
    p_rds_iam_role   =>
'arn:aws:iam::<YOUR_AWS_ACCOUNT_ID>:role/<YOUR_IAM_ROLE_NAME>'
  );
END;
/
```

3.3 Verify Upload via AWS CLI

```
aws s3 ls s3://<YOUR_S3_BUCKET>/<YOUR_S3_PREFIX>

# Expected output:
# 2026-03-18 10:23:45  204800  <YOUR_DUMP_FILENAME>
# 2026-03-18 10:23:46    1024  <YOUR_LOG_FILENAME>
```

4. Downloading from S3 to the Target RDS

4.1 Critical Signature Difference

This is the most common source of errors. Unlike upload_to_s3, the download_from_s3 procedure does NOT accept p_rds_iam_role. Including it causes ORA-06550.

Parameter	upload_to_s3	download_from_s3
p_bucket_name	YES	YES
p_directory_name	YES	YES
p_s3_prefix	YES	YES — must be EXACT full S3 key
p_prefix	YES	NO
p_rds_iam_role	YES	NO — causes ORA-06550 if included

4.2 Correct Download Block

```
-- WRONG: causes ORA-06550
-- BEGIN
--   rdsadmin.rdsadmin_s3_tasks.download_from_s3(
--     p_bucket_name    => '<YOUR_S3_BUCKET>',
--     p_s3_prefix      => '<YOUR_S3_PREFIX><YOUR_DUMP_FILENAME>',
--     p_directory_name => 'DATA_PUMP_DIR',
--     p_rds_iam_role   => 'arn:...'   -- REMOVE THIS LINE
--   );
-- END;

-- CORRECT:
BEGIN
  rdsadmin.rdsadmin_s3_tasks.download_from_s3(
    p_bucket_name    => '<YOUR_S3_BUCKET>',
    p_s3_prefix      => '<YOUR_S3_PREFIX><YOUR_DUMP_FILENAME>',
    p_directory_name => 'DATA_PUMP_DIR'
  );
END;
/
```

4.3 Confirm File Arrived

```
SELECT *
FROM   TABLE(rdsadmin.rds_file_util.listdir('DATA_PUMP_DIR'))
ORDER  BY filename;
```

SILENT FAILURE: If p_s3_prefix does not exactly match the full S3 object path including subfolder, the call succeeds with no error but NO file appears in DATA_PUMP_DIR. Always verify with listdir.

5. Importing the Table

5.1 TABLE_EXISTS_ACTION Options

Option	Behavior	Best Used When
SKIP	Leaves the existing table untouched	You want to preserve existing data
APPEND	Inserts rows without touching existing data	Merging from multiple sources
TRUNCATE	Clears all rows first, then loads	Refreshing data, keeping structure
REPLACE	Drops and recreates the table including DDL	Full replacement with structure change

5.2 Import Block

```
DECLARE
  v_hdnl NUMBER;
BEGIN
  v_hdnl := DBMS_DATAPUMP.OPEN(
    operation => 'IMPORT',
    job_mode  => 'TABLE',
    job_name  => null
  );

  DBMS_DATAPUMP.ADD_FILE(
    handle    => v_hdnl,
    filename  => '<YOUR_DUMP_FILENAME>',
    directory => 'DATA_PUMP_DIR',
    filetype  => dbms_datapump.ku$_file_type_dump_file
  );

  DBMS_DATAPUMP.ADD_FILE(
    handle    => v_hdnl,
    filename  => 'IMPORT_<YOUR_LOG_FILENAME>',
    directory => 'DATA_PUMP_DIR',
    filetype  => dbms_datapump.ku$_file_type_log_file
  );
```

```
  -- Decide what happens if table already exists
  DBMS_DATAPUMP.SET_PARAMETER(
    handle => v_hdnl,
    name   => 'TABLE_EXISTS_ACTION',
    value  => 'REPLACE'   -- or SKIP / APPEND / TRUNCATE
  );

  -- Optional: remap to a different target schema
  DBMS_DATAPUMP.METADATA_REMAP(
    handle    => v_hdnl,
    name      => 'REMAP_SCHEMA',
    old_value => '<SOURCE_SCHEMA>',
    value     => '<TARGET_SCHEMA>'
  );

  DBMS_DATAPUMP.START_JOB(v_hdnl);
END;
/
```

6. Advanced Options

6.1 Export Data Only (No DDL)

```
-- 0 = data only, 1 = include DDL (default)
DBMS_DATAPUMP.SET_PARAMETER(
  handle => v_hdnl,
  name   => 'INCLUDE_METADATA',
  value  => 0
);
```

6.2 Export a Subset of Rows

```
-- Only export rows matching a WHERE clause
DBMS_DATAPUMP.DATA_FILTER(
  handle      => v_hdnl,
  name        => 'SUBQUERY',
  value       => 'WHERE created_date >= SYSDATE - 90',
  table_name  => '<YOUR_TABLE_NAME>',
  schema_name => '<SOURCE_SCHEMA>'
);
```

6.3 Check Exact Procedure Signature on Your RDS Version

AWS updates RDS minor versions, procedure signatures can vary. Always verify what parameters your instance supports:

```
SELECT argument_name, position, data_type, in_out, defaulted
FROM   all_arguments
WHERE  object_name  = 'DOWNLOAD_FROM_S3'
AND    package_name = 'RDSADMIN_S3_TASKS'
ORDER  BY position;
```

6.4 Clean Up Orphaned Jobs

```
-- Find jobs not running (orphaned after a failure)
SELECT owner_name, job_name, state
FROM   dba_datapump_jobs
WHERE  state = 'NOT RUNNING';

-- Attach and stop the orphaned job
DECLARE
  v_hdnl NUMBER;
BEGIN
  v_hdnl := DBMS_DATAPUMP.ATTACH('SYS_EXPORT_TABLE_01', '<SOURCE_SCHEMA>');
  DBMS_DATAPUMP.STOP_JOB(v_hdnl, 1);
END;
/
```

7. Common Errors & Fixes

Error 1 — ORA-06550: Wrong number or types of arguments

```
ORA-06550: line 2, column 3:
PLS-00306: wrong number or types of arguments in call to 'DOWNLOAD_FROM_S3'
```

Cause: p_rds_iam_role was passed to download_from_s3. This parameter is not valid for the download procedure.

Fix: Remove p_rds_iam_role entirely. The IAM role is pre-configured at instance level via Options Group -> S3_INTEGRATION.

Error 2 — Access Denied (403) on Upload or Download

```
ORA-29283: invalid file operation
-- or --
An error occurred (AccessDenied) when calling the PutObject operation
```

Work through this checklist:

- IAM role is not attached to the RDS instance, go to RDS Console > your instance > Connectivity & Security > Manage IAM Roles

Role is attached but Status = PENDING — wait a few minutes for it to become ACTIVE

- Role policy is missing s3:PutObject (upload) or s3:GetObject / s3:ListBucket (download)
- Trust policy does not include rds.amazonaws.com as a trusted service
- S3 bucket is in a different region, run: aws s3api get-bucket-location --bucket <YOUR_S3_BUCKET>

Error 3 — Download Succeeds but No File Appears in DATA_PUMP_DIR

Symptom: The block runs without error, but listdir shows no new file.

Cause: The p_s3_prefix does not exactly match the full S3 object key including folder path.

```
-- Wrong: missing subfolder path
p_s3_prefix => '<YOUR_DUMP_FILENAME>'

-- Correct: full path from bucket root
p_s3_prefix => '<YOUR_S3_PREFIX><YOUR_DUMP_FILENAME>'
```

Error 4 — ORA-31634: Job Already Exists

```
ORA-31634: job already exists
```

Cause: A previous job was not cleaned up after a failure.

Fix: Attach to and stop the orphaned job as shown in Section 6.4, then re-run.

Error 5 — ORA-39001 / ORA-39000: Bad Dump File

```
ORA-39001: invalid argument value
ORA-39000: bad dump file specification
```

Cause: The .dmp file does not exist in DATA_PUMP_DIR at import time.

- Verify the file arrived: SELECT * FROM TABLE(rdsadmin.rds_file_util.listdir('DATA_PUMP_DIR'))
- If missing, re-run the download_from_s3 block with the corrected p_s3_prefix
- Check filename casing — Oracle directory file operations are case-sensitive

Error 6 — Table Already Exists Warning During Import

```
ORA-39152: Table "<SCHEMA>"."<TABLE>" exists.
Data will be appended to existing table but all dependent
metadata will be skipped due to table_exists_action of append
```

Cause: TABLE_EXISTS_ACTION was not set, defaulting to SKIP which silently ignores DDL.

Fix: Always set TABLE_EXISTS_ACTION explicitly — use REPLACE for a clean overwrite.

Error 7 — IAM Role ARN Not Found

```
An error occurred (NoSuchEntity) when calling the GetRole operation:
The role with name <YOUR_IAM_ROLE_NAME> cannot be found.
```

Cause: Typo in the role name or wrong AWS account ID in the ARN.

Fix: Get the exact ARN directly from AWS:

```
aws iam get-role \
  --role-name <YOUR_IAM_ROLE_NAME> \
  --query 'Role.Arn' \
  --output text
```

Error 8 — Bucket Does Not Exist or Wrong Region

```
An error occurred (NoSuchBucket): The specified bucket does not exist
```

Fix: Create the bucket and ensure it is in the same region as your RDS:

```
# Create bucket (all regions except us-east-1)
aws s3api create-bucket \
  --bucket <YOUR_S3_BUCKET> \
  --region <YOUR_AWS_REGION> \
  --create-bucket-configuration LocationConstraint=<YOUR_AWS_REGION>

# Verify region matches your RDS
aws s3api get-bucket-location --bucket <YOUR_S3_BUCKET>
```

8. Verification Checklist

Run through this checklist before each export/import cycle to avoid the most common issues.

- IAM role ARN is correct ,verified with aws iam get-role
- IAM role is ACTIVE (not PENDING) on the RDS instance
- S3 bucket and RDS instance are in the SAME AWS region
- Role trust policy allows rds.amazonaws.com to assume the role
- Role permission policy includes s3:PutObject (upload) and s3:GetObject + s3:ListBucket (download)
- Export job is fully complete before starting S3 upload, check dba_datapump_jobs and log file
- p_s3_prefix in download_from_s3 includes the full S3 path including subfolder
- File confirmed present in DATA_PUMP_DIR via listdir before attempting import
- TABLE_EXISTS_ACTION is set intentionally ,not left at default
- Schema remap is configured if importing into a different target schema